

Testes de API com Postman e IA

Validação de endpoints, respostas, headers e scripts automáticos no Postman

Objetivo do material

Este material orienta a criação de testes de API usando Postman, com apoio de IA para gerar scripts de validação em JavaScript. O foco é configurar requisições corretamente, validar respostas e interpretar falhas comuns, como método HTTP incorreto, body inválido e ausência de Content-Type.

O que será praticado	Resultado esperado
Configuração de requisições no Postman	Método, URL, headers e body corretamente definidos
Validação de respostas da API	Status code e campos obrigatórios verificados
Geração de scripts com IA	Scripts Post-response usando <code>pm.test()</code> , <code>pm.response.to.have.status()</code> e <code>pm.expect()</code>
Interpretação de falhas	Diagnóstico de erros como 401, 405 e 415

1. Contexto da prática

Nesta atividade, você irá validar endpoints de uma API local usando o Postman. O objetivo é entender como uma requisição HTTP é montada, como a API responde e como scripts automáticos podem verificar se o contrato da API foi respeitado.

A prática será dividida em três tipos de validação: login, consulta de dados e criação de registro. Em cada caso, a configuração correta do método, do body e dos headers será essencial para evitar erros de execução.

Endpoint	Método	Objetivo	Validação principal
/login	POST	Autenticar usuário	Status 200, token e user.email
/login	POST	Validar falha de autenticação	Status 401 e mensagem de erro
/produtos	GET	Listar produtos	Status 200 e retorno da lista
/tickets	POST	Criar ticket	Status 201 e campos do ticket criado

2. Conceitos essenciais de testes de API

Testes de API validam a comunicação entre cliente e servidor sem depender da interface visual. Em vez de clicar em telas, a validação ocorre por meio de requisições HTTP enviadas diretamente para os endpoints.

Elemento	Função no teste de API
Método HTTP	Define a intenção da requisição, como buscar, criar, atualizar ou remover dados.
URL/Endpoint	Indica qual recurso da API será acessado.
Headers	Enviam metadados da requisição, como Content-Type: application/json.
Body	Contém os dados enviados para a API em métodos como POST, PUT e PATCH.
Status code	Indica o resultado da requisição, como 200, 201, 400, 401, 404 ou 500.
Response body	Contém os dados retornados pela API, como token, usuário, lista ou mensagem de erro.

Status	Significado	Exemplo de uso
200	Sucesso	Login realizado ou listagem retornada
201	Criado	Ticket criado com sucesso

400	Requisição inválida	Campo obrigatório ausente
401	Não autorizado	Credenciais inválidas
404	Não encontrado	Rota inexistente
405	Método não permitido	Usar GET em endpoint que espera POST
415	Tipo de mídia não suportado	Enviar body sem JSON ou sem Content-Type correto

3. Preparação do ambiente

Antes de iniciar os testes, garanta que a API local esteja em execução e que o Postman consiga enviar requisições para o endereço local da aplicação.

1. Abra o Postman e aceite os termos iniciais, se solicitado, clicando em Agree and Continue.
2. Crie uma nova requisição usando o botão + no topo da interface.
3. Configure o método HTTP adequado para cada endpoint.
4. Informe a URL da API local.
5. Quando houver body, selecione Body -> raw -> JSON.
6. Verifique se o header Content-Type está definido como application/json.
7. Envie a requisição com Send e analise a resposta retornada pela API.

Atenção

Para endpoints POST que recebem JSON, configurar apenas a URL não é suficiente. É necessário selecionar Body -> raw -> JSON e confirmar o header Content-Type: application/json.

4. Atividade guiada 1 - Login com sucesso

Nesta etapa, será configurada uma requisição de login válida. O objetivo é verificar se a API autentica o usuário corretamente e retorna um token de acesso junto com os dados do usuário.

Campo	Valor
Método	POST
URL	http://127.0.0.1:3000/login
Body	raw -> JSON
Resultado esperado	200 OK

Body da requisição

```
{
  "email": "user@test.com",
  "senha": "123456"
}
```

Resposta esperada

```
{
  "token": "abc123",
  "user": {
    "email": "user@test.com"
  }
}
```

Depois de clicar em Send, confirme se o status retornado é 200 OK e se a resposta contém o campo token e o objeto user com o email esperado.

5. Scripts Post-response no Postman

Os scripts Post-response permitem automatizar as validações da resposta. Eles são executados depois que a API responde e aparecem na aba Test Results.

Área do Postman	Função
Body	Define os dados enviados na requisição.
Headers	Define metadados como Content-Type.
Scripts -> Post-response	Executa validações depois da resposta da API.
Test Results	Mostra quais validações passaram ou falharam.

Script de validação para o login

```
pm.test("Status code deve ser 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Resposta deve conter token", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("token");
});

pm.test("Email do usuário deve ser retornado", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData.user.email).to.eql("user@test.com");
});
```

8. Clique na aba Scripts da requisição.
9. Selecione Post-response.
10. Cole o script de validação.
11. Clique novamente em Send.
12. Abra Test Results e confirme se todos os testes passaram.

6. Uso da IA para gerar scripts de validação

A IA pode acelerar a criação dos scripts, mas o prompt precisa informar claramente o endpoint, o body enviado, o status esperado e os campos que devem ser validados na resposta.

Prompt recomendado

Gere scripts de validação para o Postman usando JavaScript.

Requisitos:

- Use pm.test()
- Use pm.response.to.have.status()
- Use pm.expect()
- Gere apenas o código para Scripts > Post-response

Contexto:

Estou testando o endpoint POST /login.

URL:

http://127.0.0.1:3000/login

Body enviado:

```
{
  "email": "user@test.com",
  "senha": "123456"
}
```

Resposta esperada:

- Status code 200
- Campo token presente

- Campo user.email igual a "user@test.com"

Parte do prompt	Por que é importante
Endpoint e método	Evita que a IA gere validações para a rota errada.
Body enviado	Permite validar se a resposta corresponde aos dados usados no teste.
Status esperado	Define o primeiro critério de sucesso da requisição.
Campos obrigatórios	Impede que o teste valide apenas o status e ignore o contrato da resposta.
Formato de saída	Garante que a IA retorne apenas o código para colar no Postman.

7. Atividade guiada 2 - Login inválido

Nesta etapa, o objetivo é validar o comportamento da API quando as credenciais estão incorretas. Esse teste é importante porque autenticação não deve validar apenas o cenário de sucesso.

Campo	Valor
Método	POST
URL	http://127.0.0.1:3000/login
Body	raw -> JSON
Resultado esperado	401 Unauthorized

Body com senha inválida

```
{
  "email": "user@test.com",
  "senha": "senha_errada"
}
```

Resposta esperada

```
{
  "error": "Credenciais inválidas"
}
```

Script sugerido

```
pm.test("Status code deve ser 401", function () {
  pm.response.to.have.status(401);
});

pm.test("Resposta deve conter mensagem de erro", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("error");
  pm.expect(jsonData.error).to.eql("Credenciais inválidas");
});
```

8. Atividade guiada 3 - Listagem de produtos

Nesta etapa, será validado um endpoint de consulta. Diferente do login, esse endpoint usa GET e não precisa de body.

Campo	Valor
Método	GET
URL	http://127.0.0.1:3000/produtos
Body	Não utilizar
Resultado esperado	200 OK

Script sugerido

```
pm.test("Status code deve ser 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Resposta deve ser uma lista de produtos", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.be.an("array");
});

pm.test("Cada produto deve conter campos básicos", function () {
  const jsonData = pm.response.json();
  if (jsonData.length > 0) {
    pm.expect(jsonData[0]).to.have.property("id");
    pm.expect(jsonData[0]).to.have.property("nome");
  }
});
```

Ponto de atenção

Em requisições GET, normalmente não há envio de body. Se a API retornar erro, revise primeiro a URL e confirme se o servidor local está em execução.

9. Atividade guiada 4 - Criação de ticket

Nesta etapa, será criado um novo ticket usando POST. A validação deve confirmar que a API retornou status 201 Created e os campos esperados no corpo da resposta.

Campo	Valor
Método	POST
URL	http://127.0.0.1:3000/tickets
Body	raw -> JSON
Resultado esperado	201 Created

Body da requisição

```
{
  "title": "Erro ao acessar conta",
  "category": "acesso",
  "priority": "alta"
}
```

Resposta esperada

```
{
  "id": 42,
  "title": "Erro ao acessar conta",
  "category": "acesso",
  "priority": "alta",
  "createdAt": "2026-05-12T10:30:00Z"
}
```

Script sugerido

```
pm.test("Status code deve ser 201", function () {
  pm.response.to.have.status(201);
});
```

```

pm.test("Resposta deve conter os campos do ticket", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("id");
  pm.expect(jsonData).to.have.property("title");
  pm.expect(jsonData).to.have.property("category");
  pm.expect(jsonData).to.have.property("priority");
  pm.expect(jsonData).to.have.property("createdAt");
});

pm.test("Dados do ticket devem estar corretos", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData.title).to.eql("Erro ao acessar conta");
  pm.expect(jsonData.category).to.eql("acesso");
  pm.expect(jsonData.priority).to.eql("alta");
});

```

10. Interpretação de falhas comuns

Quando um teste de API falha, o primeiro passo é identificar se o problema está na requisição, na resposta da API ou no script de validação. A tabela abaixo resume erros frequentes.

Falha observada	Causa provável	O que revisar
415 Unsupported Media Type	Body não foi enviado como JSON ou header ausente	Body -> raw -> JSON e Content-Type: application/json
401 Unauthorized	Credenciais inválidas	Email, senha e regra de autenticação
404 Not Found	Endpoint incorreto ou inexistente	URL e rota configurada na API
405 Method Not Allowed	Método HTTP incorreto	GET, POST, PUT, PATCH ou DELETE
Test Results com falha	Resposta diferente do esperado	Script, status code e campos retornados

Correção do erro 415

O erro 415 indica que a API esperava receber JSON, mas o Postman não enviou a requisição como application/json. Para corrigir:

- Volte na aba Body.
- Selecione raw.
- Escolha JSON no seletor de formato.
- Confirme em Headers se existe Content-Type: application/json.
- Cole o script na área Scripts -> Post-response, não em Pre-request.
- Clique em Send novamente e verifique Test Results.

11. Desafio prático - API de Recados

Neste desafio, você irá aplicar o mesmo processo em uma API de recados. Para cada cenário, configure a requisição, envie para a API, crie um prompt contextualizado para a IA gerar o script Post-response e valide o resultado em Test Results.

Cenário	Endpoint	Resultado esperado	Campos a validar
1 - Criar recado	POST /recados	Status 201	id, autor, mensagem, lido=false
2 - Listar recados	GET /recados	Status 200	lista e campos obrigatórios
3 - Erro sem mensagem	POST /recados	Status 400	campo error

Exemplo de prompt para o Cenário 1

Gere scripts Post-response para o Postman usando JavaScript.

Requisitos:

- Use `pm.test()`
- Use `pm.response.to.have.status()`
- Use `pm.expect()`
- Gere apenas o código, sem explicações.

Endpoint:

POST `http://localhost:3000/recados`

Body enviado:

```
{
  "autor": "Ana",
  "mensagem": "Revisar o material da aula"
}
```

Valide:

- status code 201
- campo id presente
- autor igual a "Ana"
- mensagem igual a "Revisar o material da aula"
- lido igual a false

Script esperado para o Cenário 1

```
pm.test("Status code deve ser 201", function () {
  pm.response.to.have.status(201);
});

pm.test("Resposta deve conter id", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("id");
});

pm.test("Autor e mensagem devem estar corretos", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData.autor).to.eql("Ana");
  pm.expect(jsonData.mensagem).to.eql("Revisar o material da aula");
});

pm.test("Recado deve iniciar como não lido", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData.lido).to.eql(false);
});
```

Orientação para os demais cenários

Para o cenário de listagem, valide se a resposta é uma lista e se os registros possuem campos obrigatórios. Para o cenário de erro, envie uma requisição sem o campo mensagem e valide se o status é 400 e se a resposta contém o campo error.

12. Entrega esperada

- Print ou evidência da requisição POST `/login` com status 200 e Test Results passando.
- Print ou evidência da requisição POST `/login` com credenciais inválidas e status 401.
- Print ou evidência da requisição GET `/produtos` com status 200.

- Print ou evidência da requisição POST /tickets com status 201.
- Scripts Post-response usados em cada requisição.
- Breve análise dos resultados, informando quais validações passaram e quais falhas foram corrigidas.
- Para o desafio de recados: prompts contextualizados e scripts gerados para os três cenários.

13. Checklist final

Item	Verificação
API local	Está em execução antes de enviar as requisições
Método HTTP	GET ou POST configurado conforme o endpoint
Body	Configurado como raw -> JSON quando necessário
Headers	Content-Type: application/json presente nos endpoints com JSON
Status code	Validação automática implementada no script
Campos da resposta	Token, user.email, id, title, category, priority ou error validados
Scripts	Colados em Scripts -> Post-response
Test Results	Resultados analisados após clicar em Send
IA	Código gerado revisado antes de ser aceito

Síntese

Testes de API com Postman não verificam apenas se a API respondeu. Eles validam o contrato da comunicação: método, endpoint, headers, body, status code e campos retornados. A IA pode acelerar a criação dos scripts, mas a revisão crítica continua sendo obrigatória.